



**K.R. MANGALAM UNIVERSITY**  
THE COMPLETE WORLD OF EDUCATION

**Operation System Assignment**

**Subject: Operating System**

**5th Semester**

**Faculty Name: Dr. Saneh Lata Yadav**

**Submitted By: Utsav Ratan**

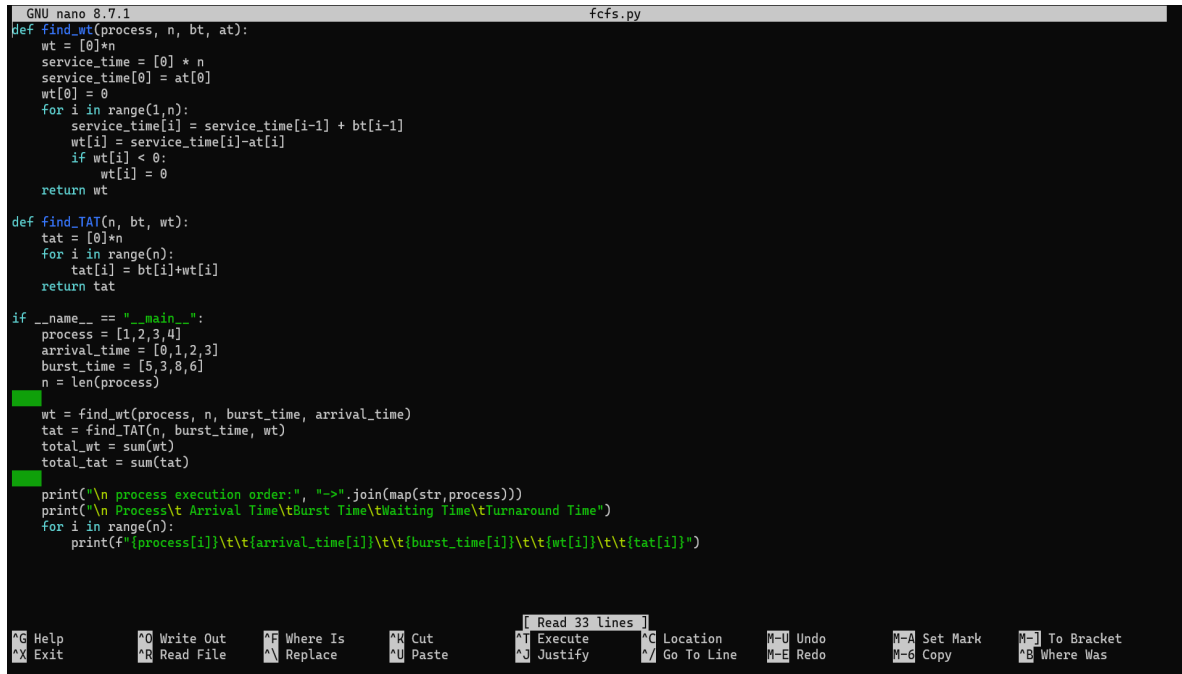
**Roll No: 2401010046**

**Section: B. Tech CSE Core (B)**

# Experiment 1 – First Come, First Served (FCFS)

## Description: -

This experiment implements the non-preemptive First Come, First Served (FCFS) CPU scheduling algorithm. Processes are executed in arrival order. The program calculates the waiting time and turnaround time for each process using the given arrival and burst times.



```
GNU nano 8.7.1                                fcfs.py
def find_wt(process, n, bt, at):
    wt = [0]*n
    service_time = [0] * n
    service_time[0] = at[0]
    wt[0] = 0
    for i in range(1,n):
        service_time[i] = service_time[i-1] + bt[i-1]
        wt[i] = service_time[i]-at[i]
        if wt[i] < 0:
            wt[i] = 0
    return wt

def find_TAT(n, bt, wt):
    tat = [0]*n
    for i in range(n):
        tat[i] = bt[i]+wt[i]
    return tat

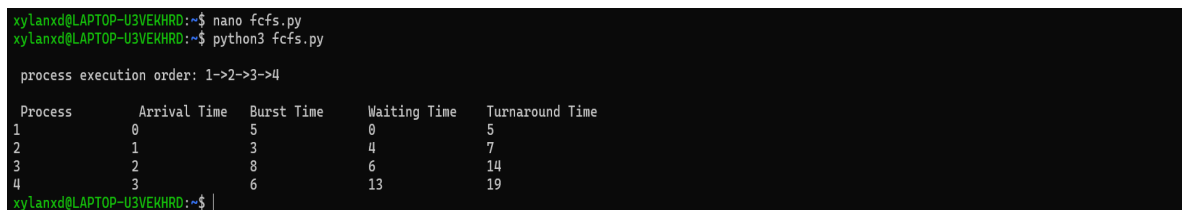
if __name__ == "__main__":
    process = [1,2,3,4]
    arrival_time = [0,1,2,3]
    burst_time = [5,3,8,6]
    n = len(process)

    wt = find_wt(process, n, burst_time, arrival_time)
    tat = find_TAT(n, burst_time, wt)
    total_wt = sum(wt)
    total_tat = sum(tat)

    print("\n process execution order:", "~>".join(map(str,process)))
    print("\n Process\t Arrival Time\tBurst Time\tWaiting Time\tTurnaround Time")
    for i in range(n):
        print(f"{process[i]}\t\t{arrival_time[i]}\t\t{burst_time[i]}\t\t{wt[i]}\t\t{tat[i]}")

[ Read 33 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^A Set Mark   ^_ To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line ^E Redo       ^M Copy       ^B Where Was
```

Screenshot 1



```
xylanxd@LAPTOP-U3VEKHRD:~$ nano fcfs.py
xylanxd@LAPTOP-U3VEKHRD:~$ python3 fcfs.py

process execution order: 1->2->3->4

Process      Arrival Time  Burst Time   Waiting Time  Turnaround Time
1             0             5            0            5
2             1             3            4            7
3             2             8            6            14
4             3             6            13           19
xylanxd@LAPTOP-U3VEKHRD:~$
```

Screenshot 2

## Experiment 2 – Shortest Job First (SJF)

### Description: -

This experiment implements non-preemptive Shortest Job First (SJF) CPU scheduling. At each selection point, the program chooses the arrived, unfinished process with the shortest burst time. It calculates waiting time and turnaround time and displays the execution order and averages.

```
GNU nano 8.7.1 sjf.py
def find_wt(process, n, bt, at):
    wt = [0] * n
    complete = [False] * n
    current_time = 0
    completed_count = 0

    while completed_count < n:
        min_bt = float('inf')
        short_idx = -1

        for i in range(n):
            if at[i] <= current_time and not complete[i] and bt[i] < min_bt:
                min_bt = bt[i]
                short_idx = i

        if short_idx == -1:
            next_arrival = float('inf')
            for i in range(n):
                if not complete[i] and at[i] < next_arrival:
                    next_arrival = at[i]
            current_time = next_arrival
            continue

        wt[short_idx] = current_time - at[short_idx]
        current_time += bt[short_idx]
        complete[short_idx] = True
        completed_count += 1

    return wt

def find_TAT(n, bt, wt):
    tat = [0] * n
    for i in range(n):
        tat[i] = bt[i] + wt[i]
    return tat

if __name__ == "__main__":
    process = [1, 2, 3, 4]
    arrival_time = [0, 1, 2, 3]
    burst_time = [5, 3, 8, 6]
    n = len(process)

    wt = find_wt(process, n, burst_time, arrival_time)
    tat = find_TAT(n, burst_time, wt)
    total_wt = sum(wt)
    total_tat = sum(tat)

    finish_times = [arrival_time[i] + tat[i] for i in range(n)]
    sorted_indices = sorted(range(n), key=lambda k: finish_times[k])
    execution_order = [process[idx] for idx in sorted_indices]

    print("\n process execution order:", "->".join(map(str, execution_order)))
    print("\n Process\t Arrival Time\tBurst Time\tWaiting Time\tTurnaround Time")
    for i in range(n):
        print(f"{process[i]}\t\t{arrival_time[i]}\t\t{burst_time[i]}\t\t{wt[i]}\t\t{tat[i]}")

    print(f"\nAverage Waiting Time: {total_wt / n:.2f}")
    print(f"\nAverage Turnaround Time: {total_tat / n:.2f}")
```

Screenshot 1

```
wt = find_wt(process, n, burst_time, arrival_time)
tat = find_TAT(n, burst_time, wt)
total_wt = sum(wt)
total_tat = sum(tat)

finish_times = [arrival_time[i] + tat[i] for i in range(n)]
sorted_indices = sorted(range(n), key=lambda k: finish_times[k])
execution_order = [process[idx] for idx in sorted_indices]

print("\n process execution order:", "->".join(map(str, execution_order)))
print("\n Process\t Arrival Time\tBurst Time\tWaiting Time\tTurnaround Time")
for i in range(n):
    print(f"{process[i]}\t\t{arrival_time[i]}\t\t{burst_time[i]}\t\t{wt[i]}\t\t{tat[i]}")

print(f"\nAverage Waiting Time: {total_wt / n:.2f}")
print(f"\nAverage Turnaround Time: {total_tat / n:.2f}")
```

Screenshot 2

```
xylansd@LAPTOP-U3VEHHRD:~$ nano sjf.py
xylansd@LAPTOP-U3VEHHRD:~$ python3 sjf.py

process execution order: 1->2->4->3

Process      Arrival Time  Burst Time   Waiting Time  Turnaround Time
1             0              5             0              5
2             1              3             4              7
3             2              8            12             20
4             3              6             5             11

Average Waiting Time: 5.25
Average Turnaround Time: 10.75
```

Screenshot 3